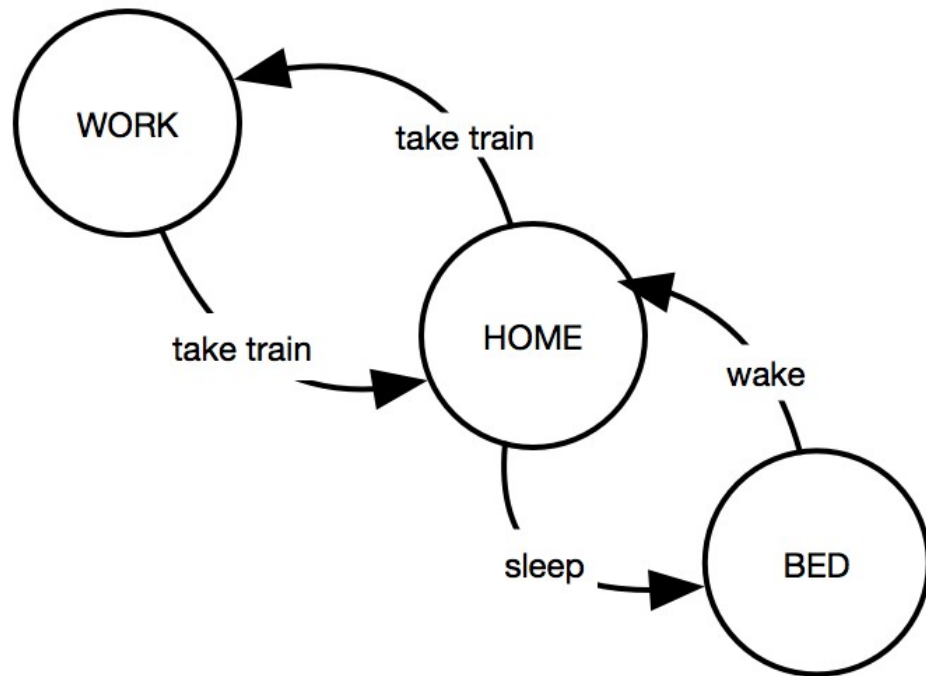


Finite automata (state machines)

A machine that consists of a set of states and a set of transition rules that dictate how the machine can move from one



The **states** in this machine are:

- **Home**
- **Work**
- **Sleep**

The **transitions** are:

- **wake**
- **sleep**
- **take train**

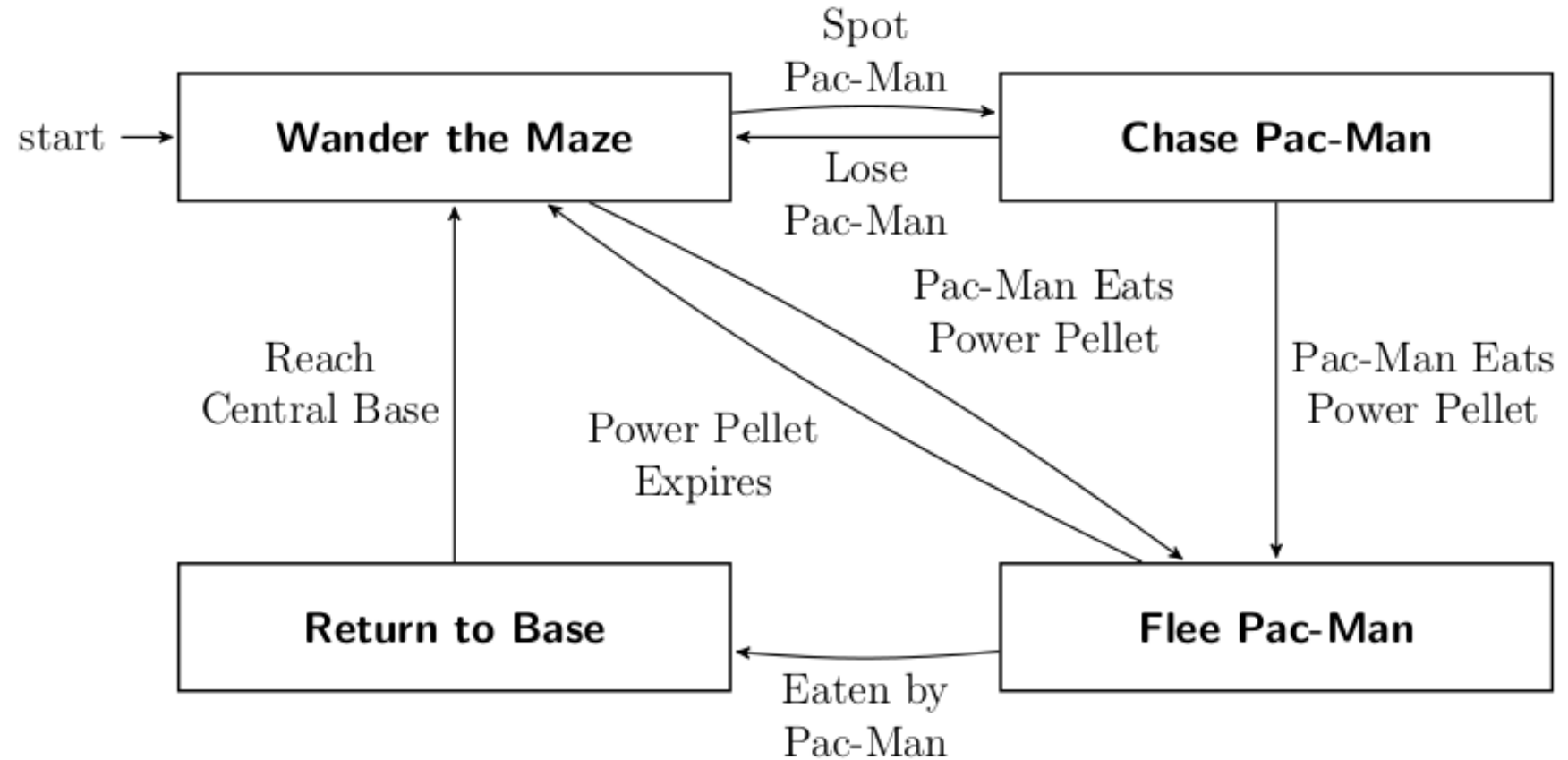
Starting state: Home

Transitions:

[take train, take train, sleep, wake, take train]

What would be the final state in this instance?

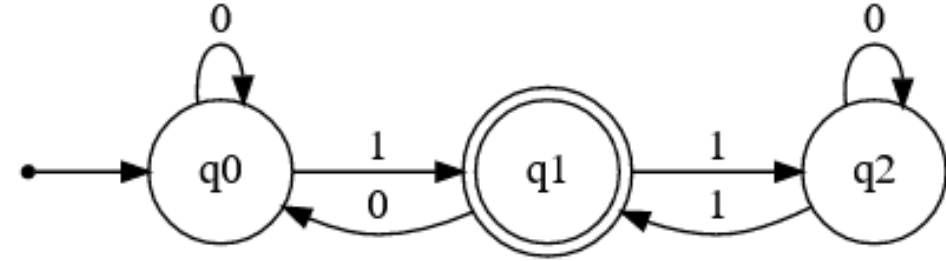
The ghosts in Pac-man operate via automata



Structure of automata with binary transition functions

An automaton is a 5 item tuple: $\{Q, \Sigma, q, F, \delta\}$

Q	Set of all states	$\{q_0, q_1, q_2\}$
Σ	Set of input symbols (input alphabet)	$\{0,1\}$
q	Initial state	q_0
F	Final state $F \subseteq Q$	q_1
δ	Transition function: $\delta = Q \times \Sigma \rightarrow Q$	$T(q_0,0) \rightarrow q_0$ $T(q_0,1) \rightarrow q_1$ $T(q_1,0) \rightarrow q_0$ $T(q_1,1) \rightarrow q_2$ $T(q_2,0) \rightarrow q_2$ $T(q_2,1) \rightarrow q_1$



How does this machine work?

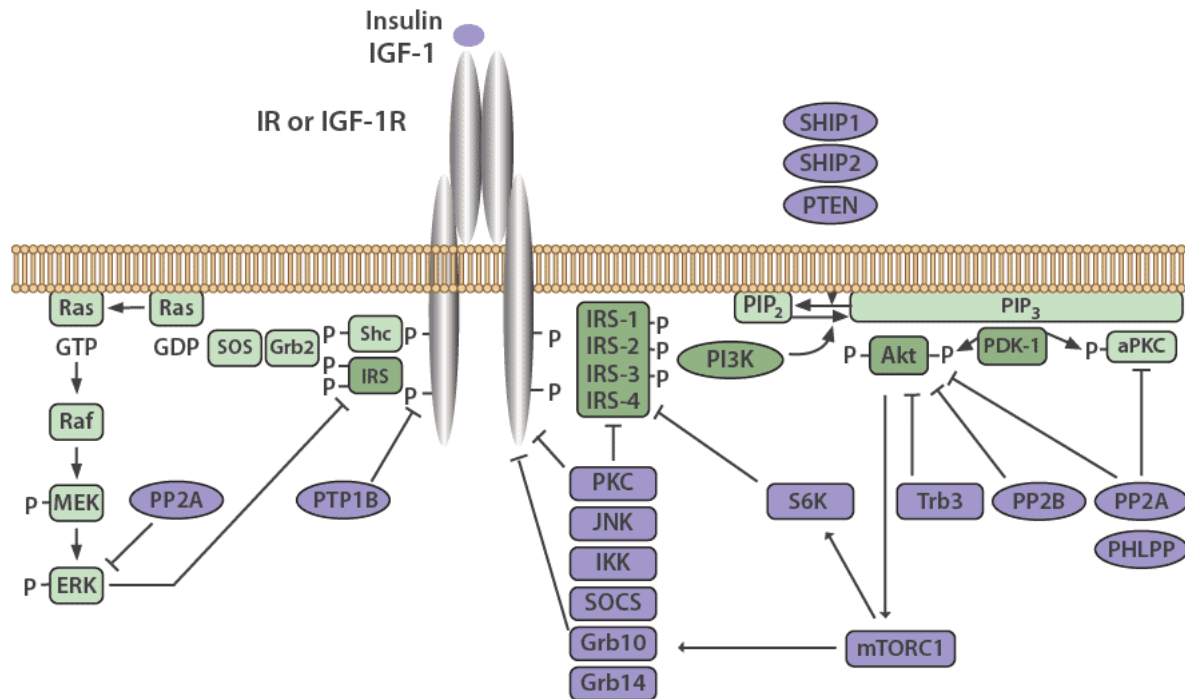
If the Input stream = 10111

Then output stream = $q_0, q_1, q_0, q_1, q_2, q_1$

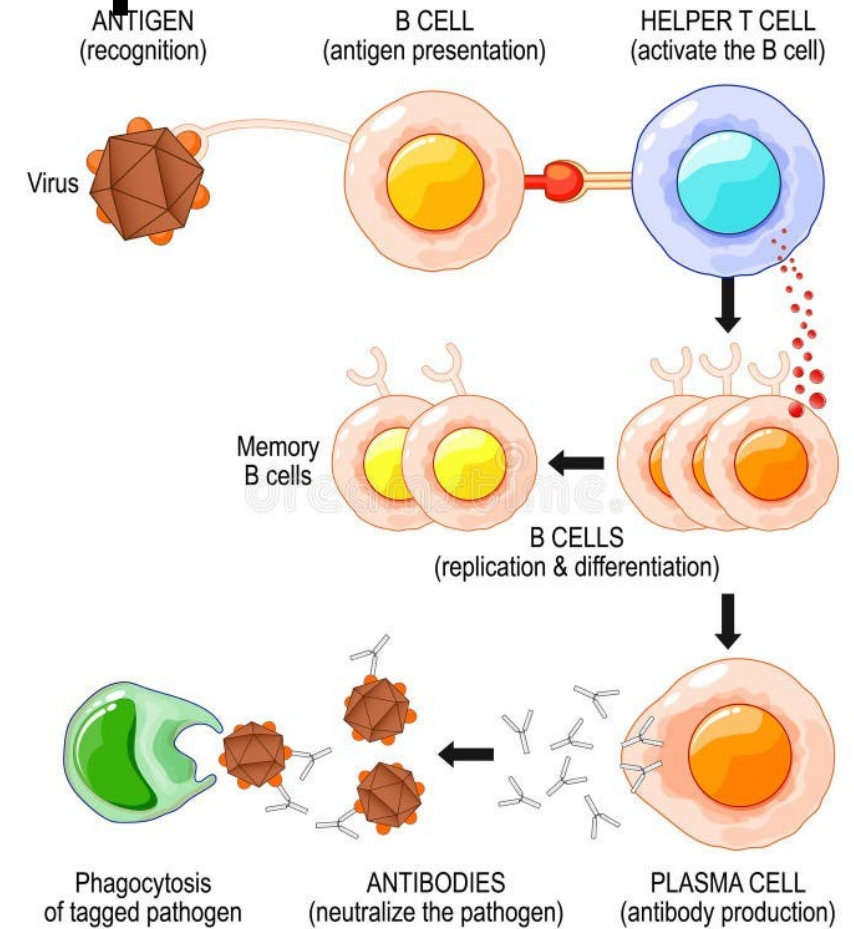
*These machines can be mutated by changing a transition function OR output symbol for a state

Can automata model biological processes?

Signal transduction



Humoral immune response



Modeling Red Queen Dynamics

We aim to develop a computational model of host-pathogen coevolution using finite automata and evolutionary game theory, which will be useful for:

- studying population dynamics during evolutionary arms races
- identifying conditions that lead to major population transitions

Key questions to investigate:

- How does adjusting model parameters influence transitions and stability of entire system?
 - generation times, mutation rates and constraints, population size, recombination
- Are microbial strains shaped by host immune landscape, specifically polyclonal antibody cross reactivity?
 - Is antigen allele diversity driven by host specificity or immune escape? Or both?

Proof-of-Concept Tests

All tests performed using binary input/outputs, population size = 40 automata, mutation rate = 0.33 per generation, and tournament selection

Test 1. Adaptation to a target output

- Evolve machines that emit a desired output, given a fixed input
- Example: Given the input '100101' and target '0001011', evolve a population of machines so that the output stream always matches the target

Test 2. Pattern recognition

- Evolve machines that classify strings based on whether they show a specified pattern or not
- Pattern = input string contains exactly three 1's
- Final output = 1 if input matches pattern, 0 if input does not match the pattern

Test 3. Pattern recognition with coevolution

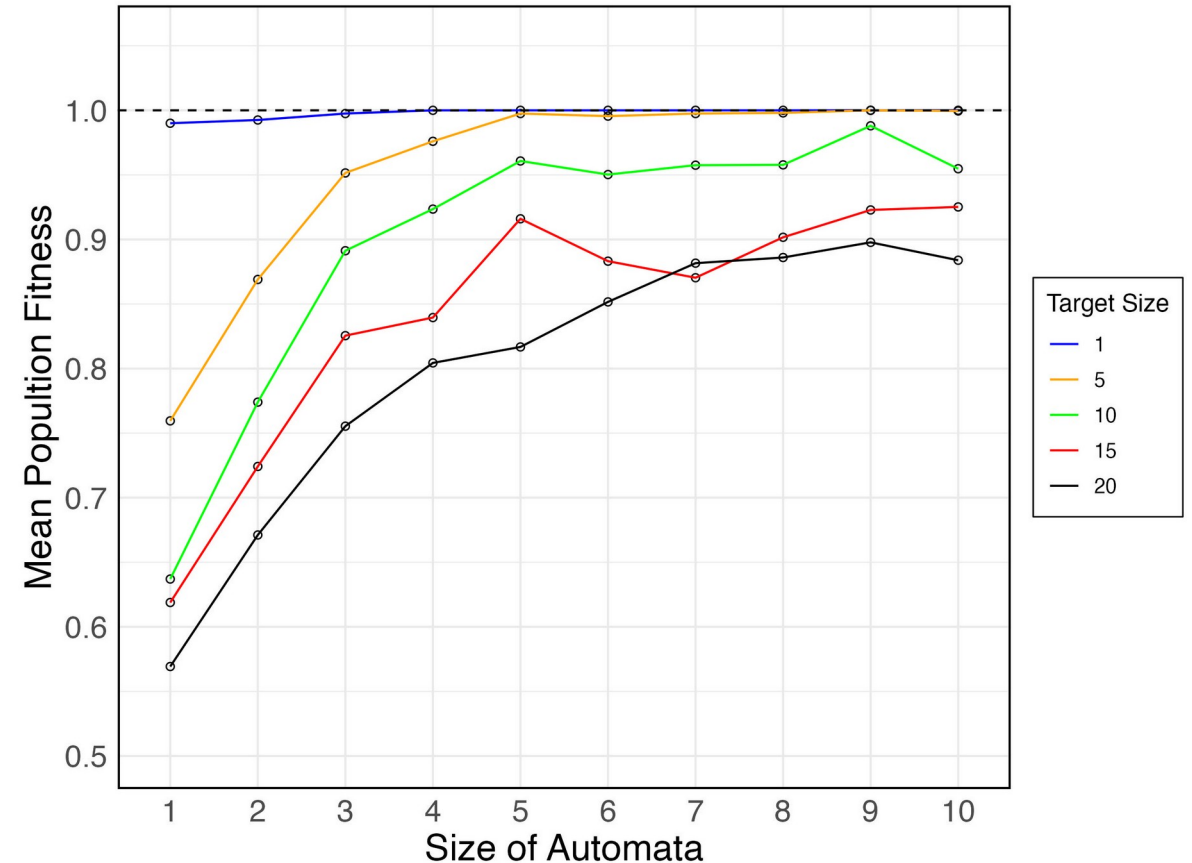
- Same as test 2, but automata and input strings coevolve during simulation
- input strings evolve to try to evade recognition by the machines
- machines evolve to try to successfully classify the input strings

Test 1: Adaptation to target output

100 simulations per target string length per automata size

Parameters:

- Target strings: $L = 1$ to 20 binary strings
- Input stream: an $L - 1$ long random binary string
- Generations: $g = 100$
- Fitness: normalized hamming distance between output and target strings



- Test gets more challenging for all automata sizes as target size increases
- Fitness increases with automata size (up to a certain point)
- Evolving for more generation (e.g. 1000) will likely produce perfect fitness scores for larger sized automata

Proof-of-Concept Tests

All tests performed using binary input/outputs, population size = 40 automata, mutation rate = 0.33 per generation, and tournament selection

Test 1. Adaptation to a target environment

- Evolve machines that emit a desired binary string, given a fixed input
- Example: Given the input '100101' and target '0001011', evolve a population of machines so that the output stream always matches the target

Test 2. Pattern recognition

- Evolve machines that classify strings based on whether they show a specified pattern or not
- Pattern = input string contains exactly three 1's
- Final output = 1 if input matches pattern, 0 if input does not match the pattern

Test 3. Pattern recognition with coevolution

- Same as test 2, but automata and input strings coevolve during simulation
- input strings evolve to try to evade recognition by the machines
- machines evolve to try to successfully classify the input strings

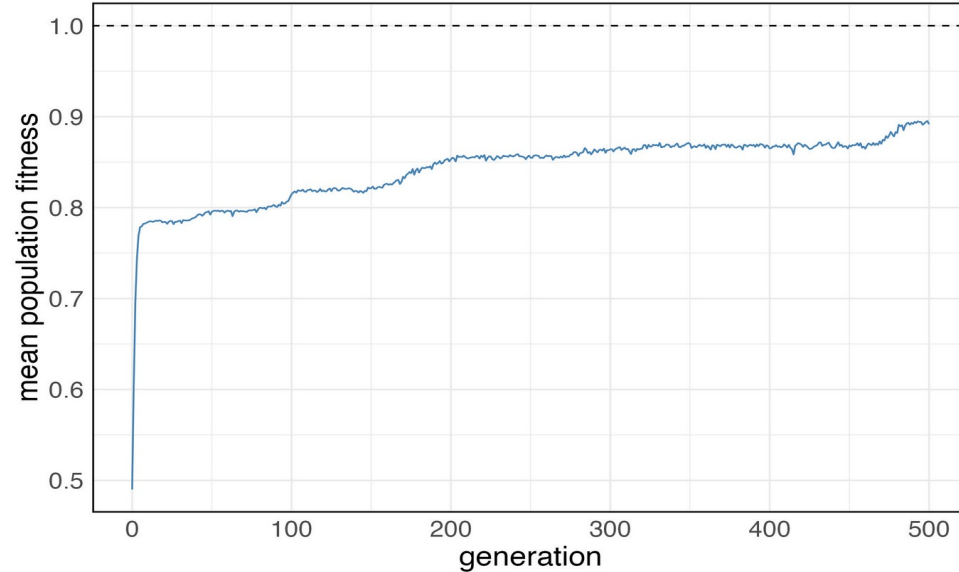
Test 2: Pattern Recognition

Training the population:

Problem set: 240 random strings, length 3-11

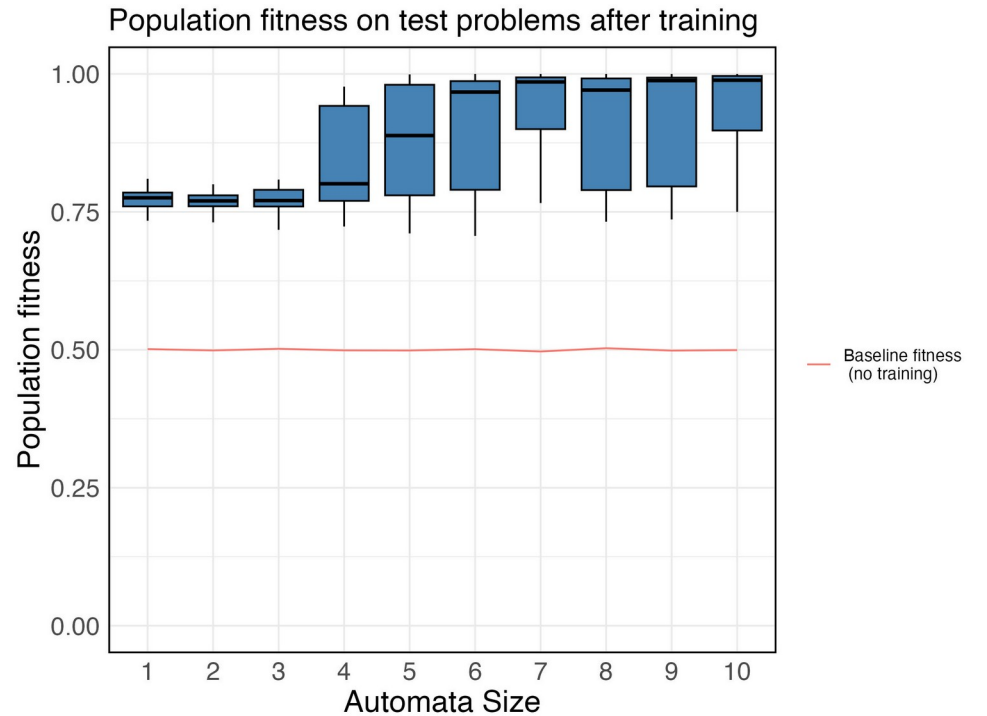
Fitness = number of strings in problem set correctly classified

5 state automata evolving for 500 generations, n = 10 simulations



Testing the population:

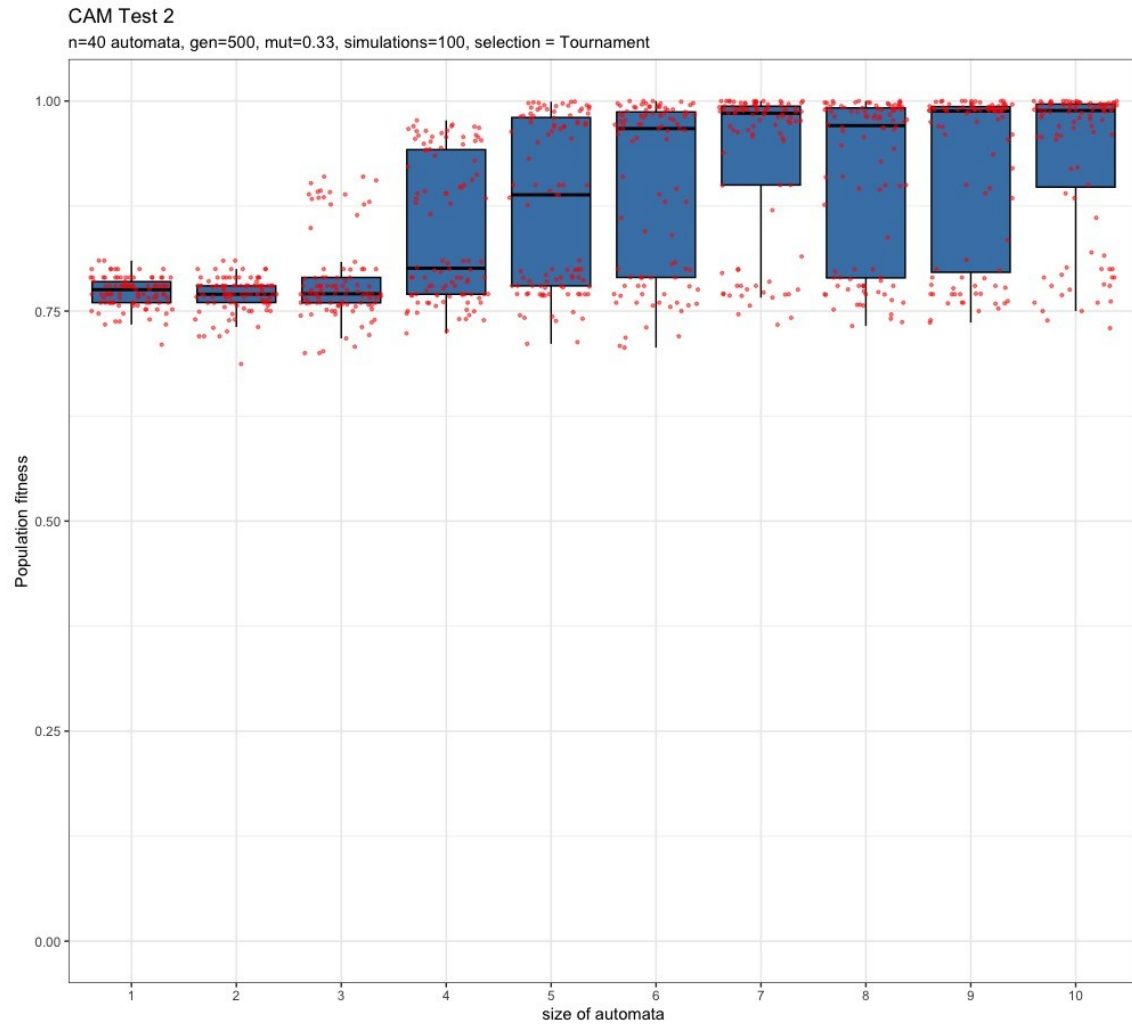
Test set: 1000 random strings, length 3-11



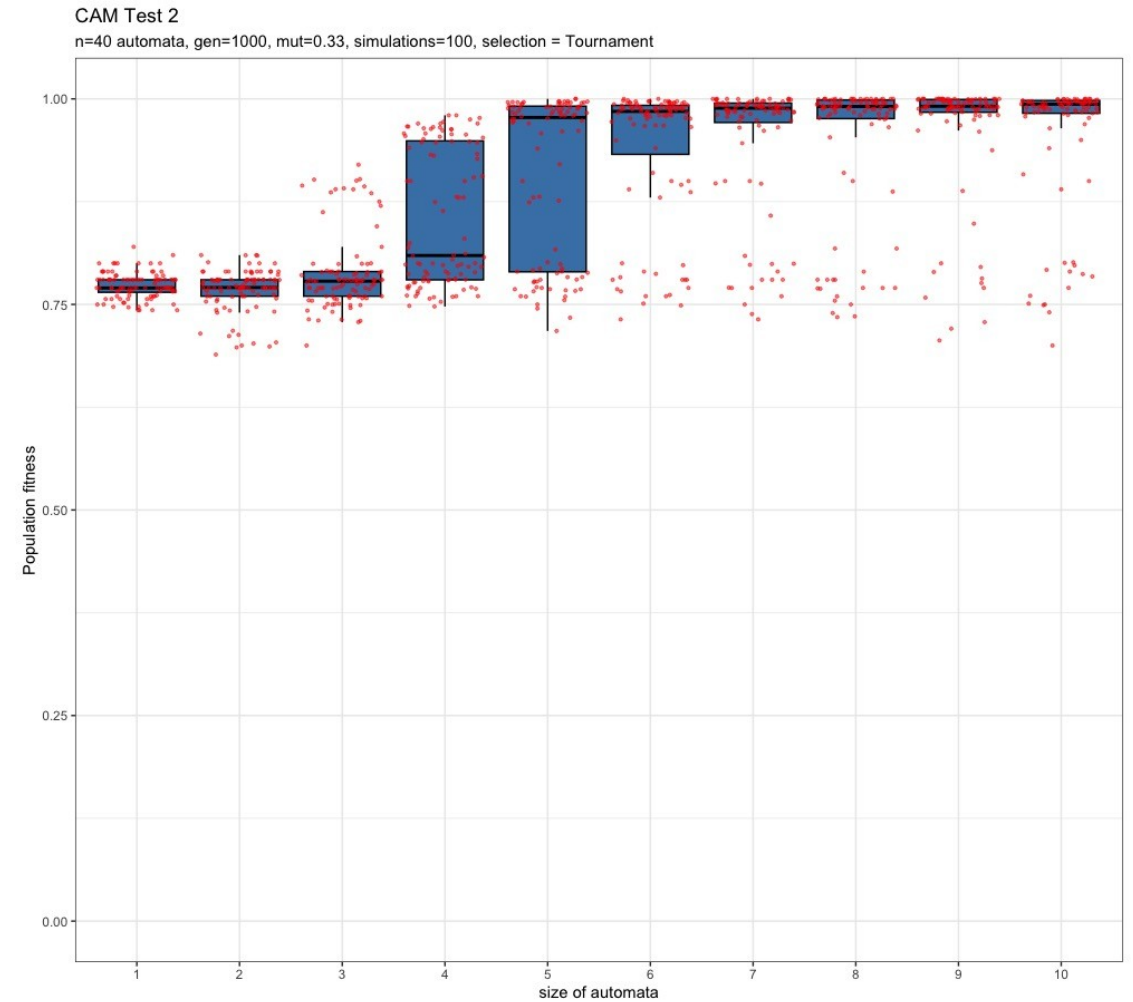
- All automata become better at recognizing pattern after training, but machines containing more states perform better.
- Population may be **overfit** to training set
- Most fit individuals in population for automata sizes 5-10 score perfectly on test set.

Test 2: Pattern Recognition

500 generation



1000
generations



Proof-of-Concept Tests

All tests performed using binary input/outputs, population size = 40 automata, mutation rate = 0.33 per generation, and tournament selection

Test 1. Adaptation to a target environment

- Evolve machines that emit a desired binary string, given a fixed input
- Example: Given the input '100101' and target '0001011', evolve a population of machines so that the output stream always matches the target

Test 2. Pattern recognition

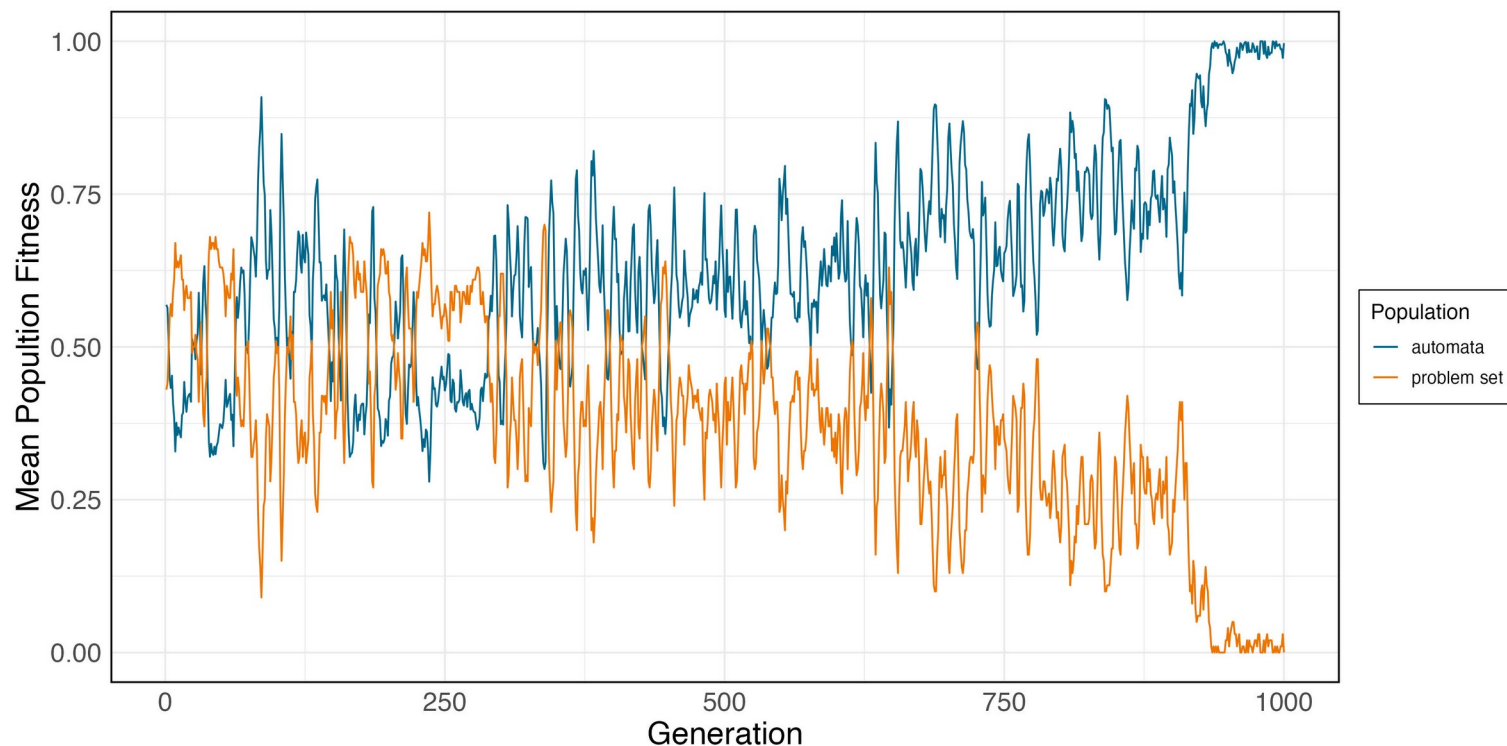
- Evolve machines that classify strings based on whether they show a specified pattern or not
- Pattern = input string contains exactly three 1's
- Final output = 1 if input matches pattern, 0 if input does not match the pattern

Test 3. Pattern recognition with coevolution

- Same as test 2, but automata and input strings coevolve during simulation
- input strings evolve to try to evade recognition by the machines
- machines evolve to try to successfully classify the input strings

Test 3: Pattern Recognition with coevolving problem set

A population of 5 state automata coevolving with a problem set (n=240 random binary strings, length 3-11)



Automata fitness =
number of strings in
problem set correctly
classified

Problem set fitness =
number of automata in
population that is
successfully evaded

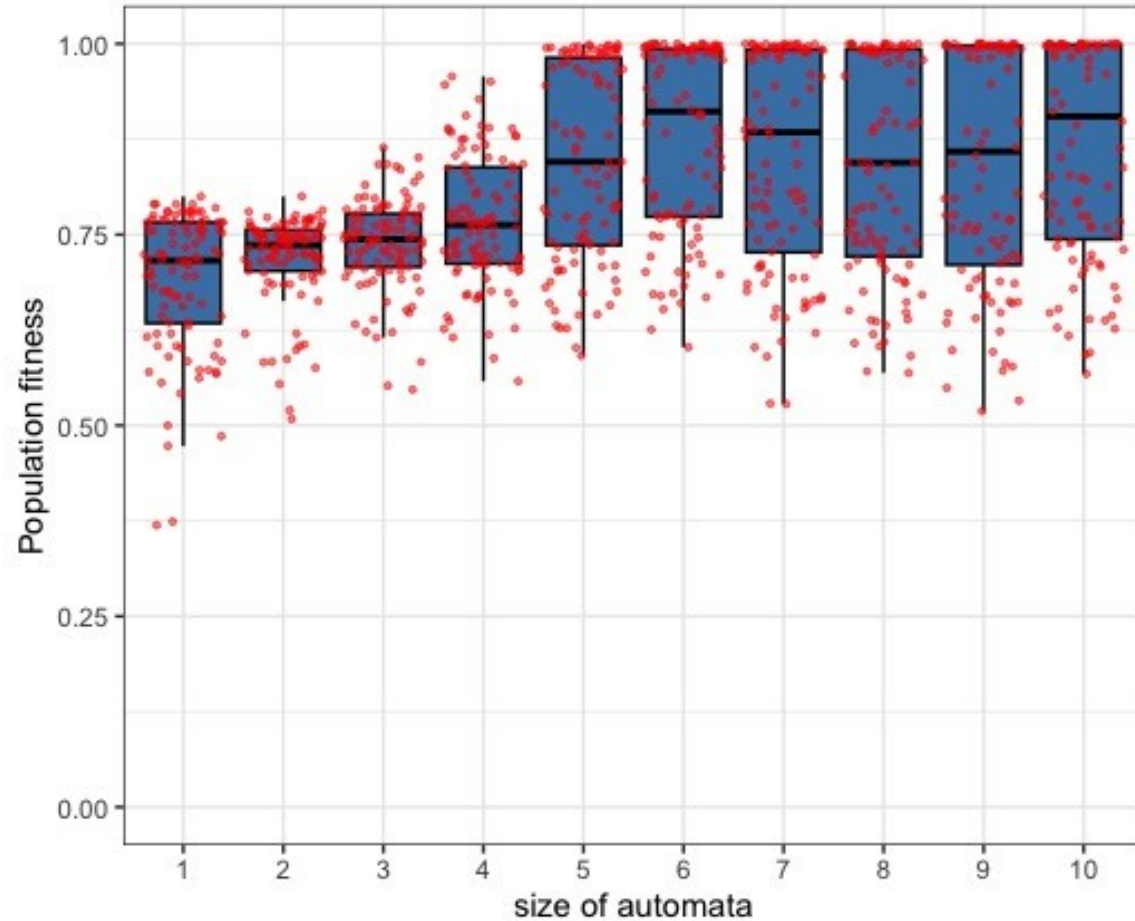
- Automata will always win if given enough time to evolve
- Number of generations until automata wins varies
- We believe that coevolution will avoid the population being overfit to the training set

Test 3: Pattern Recognition with coevolving problem set

500 generation

CAM Test 3

n=40 automata, gen=500, mut=0.33, simulations=100, selection = Tourr

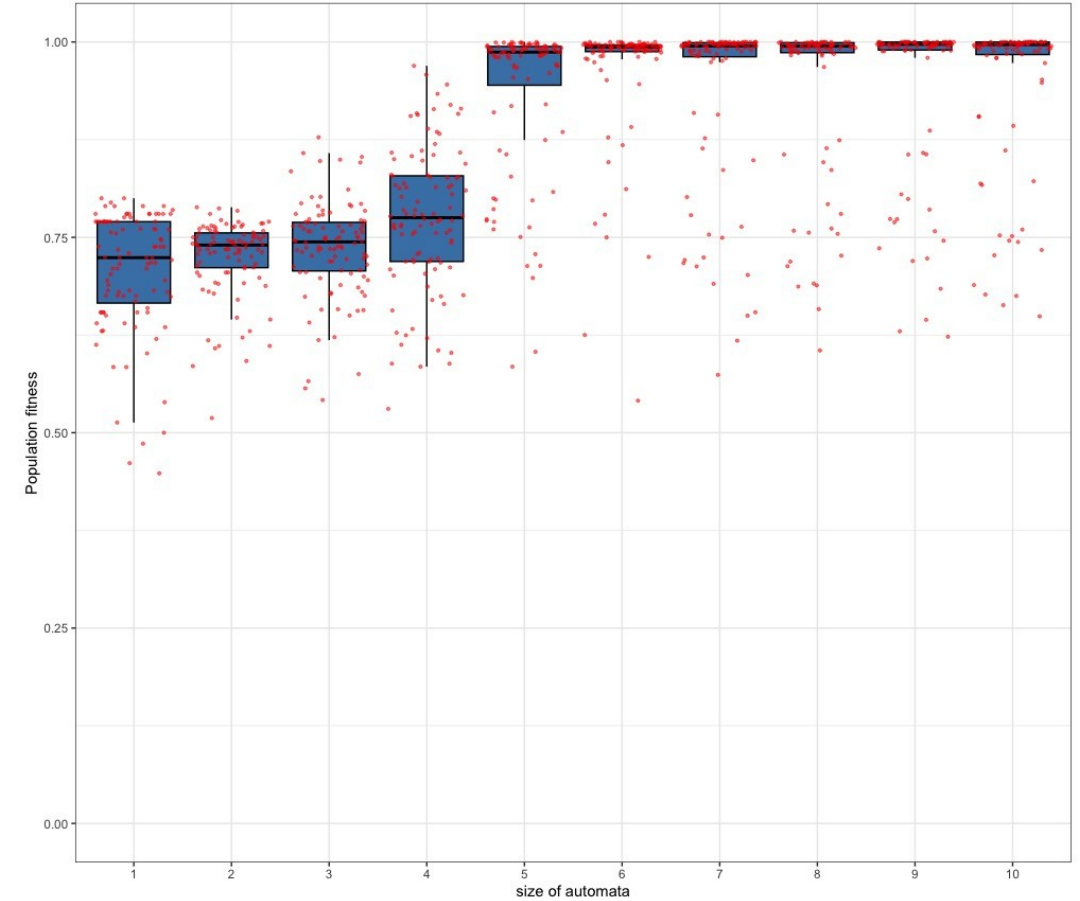


1000

generations

CAM Test 3

n=40 automata, gen=1000, mut=0.33, simulations=100, selection = Tournament



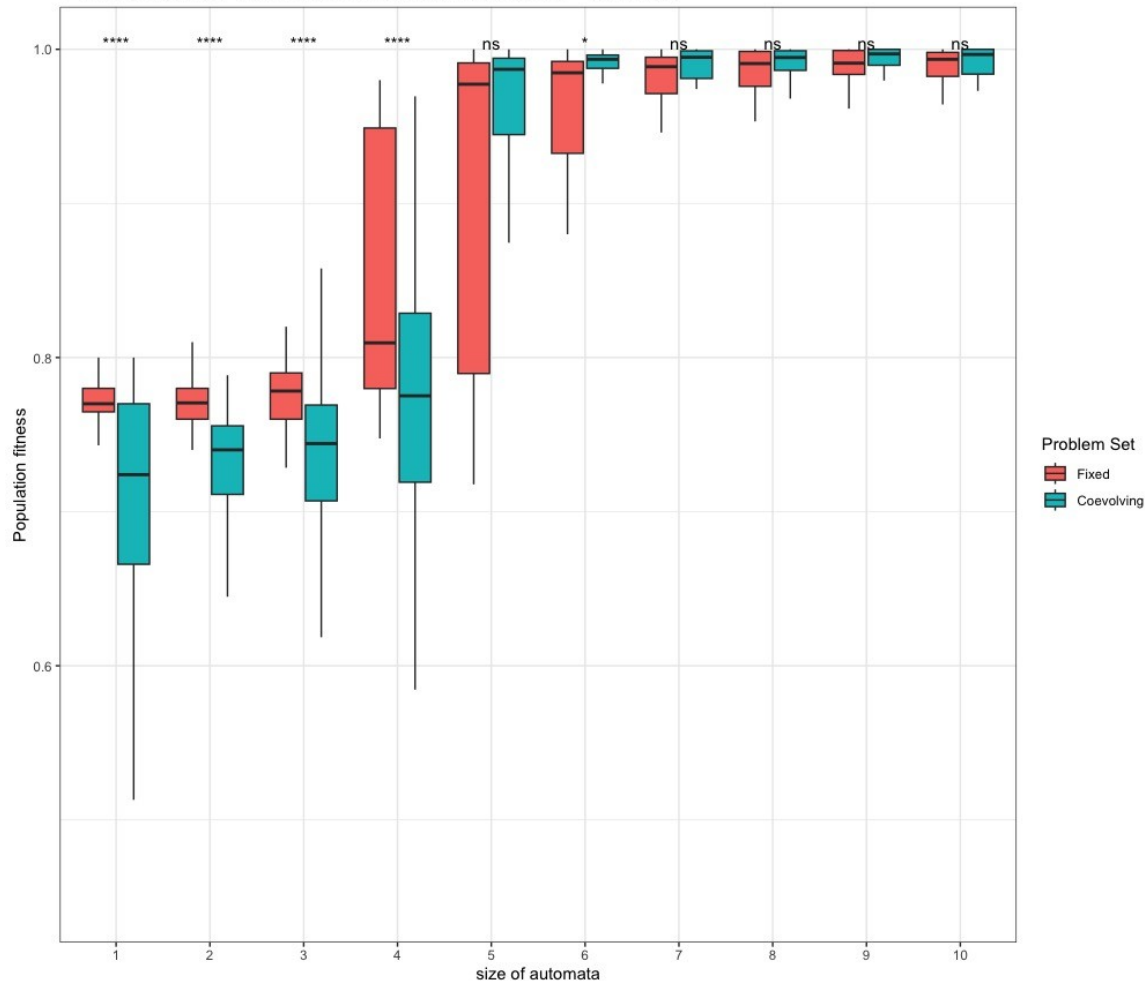
Coevolving machines have higher frequency of best performer being perfect on test set

- machines with fixed problem sets are over fitting to the training set

Fitness on test set after 1000

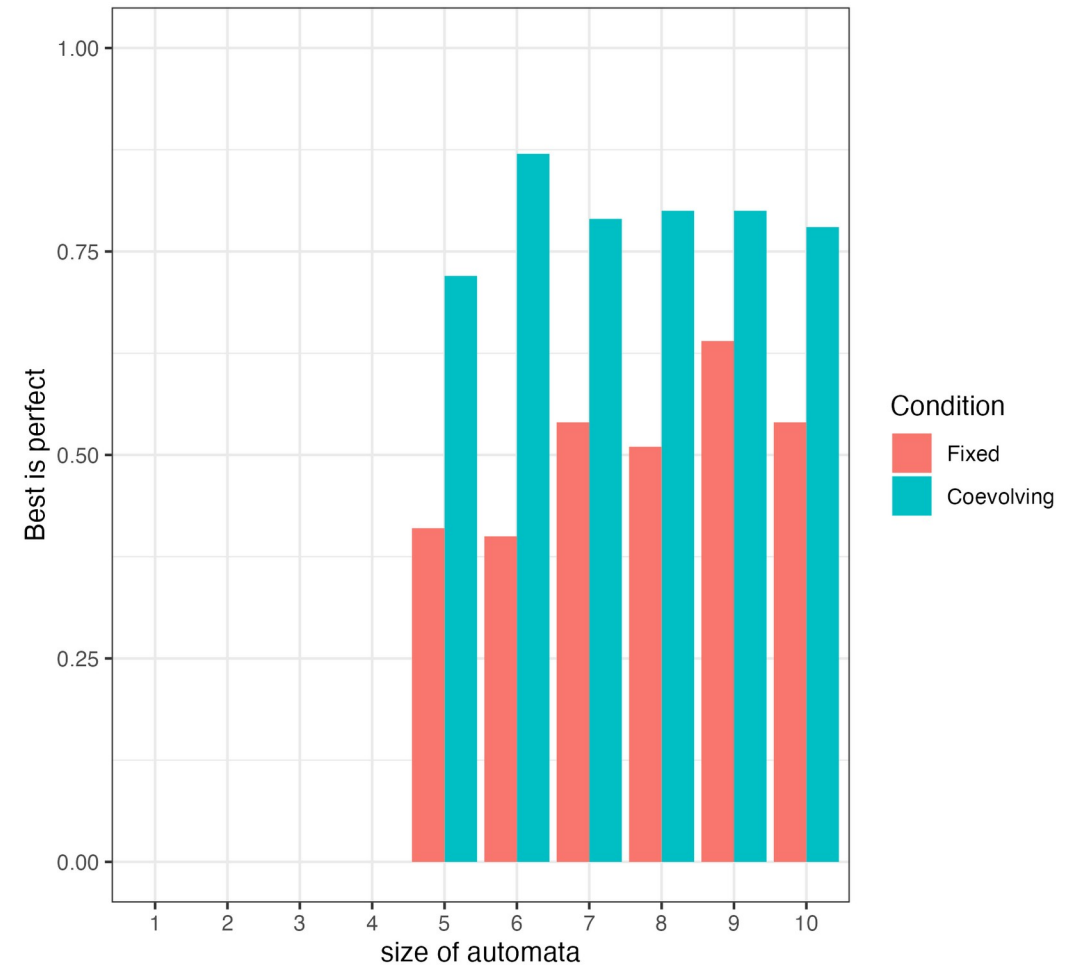
generations

n=40 automata, gens = 1000, mut=0.33, simulations=100, selection = Tournament



Frequency of best performing machine on test set being perfect

n=40 automata, gen=1000, mut=0.33, simulations=100, selection = Tournament



Next steps:

- Determine biologically relevant parameters for modeling host-pathogen coevolution
- Develop evolutionary “games” for host and pathogen populations to compete in